

Minimum Edit Distance - Example

Src\Tar	#	e	x	e	c	u	t	i	o	n
#	0	1	2	3	4	5	6	7	8	9
i	1	2	3	4	5	6	7	6	7	8
n	2	3	4	5	6	7	8	7	8	7
t	3	4	5	6	7	8	7	8	9	8
e	4	3	4	5	6	7	8	9	10	9
n	5	4	5	6	7	8	9	10	11	10
t	6	5	6	7	8	9	8	9	10	11
i	7	6	7	8	9	10	9	8	9	10
o	8	7	8	9	10	11	10	9	8	9
n	9	8	9	10	11	12	11	10	9	8

Minimum Edit Distance - Example

i n t e n t i o n	← delete i
n t e n t i o n	← substitute n by e
e t e n t i o n	← substitute t by x
e x e n t i o n	← insert u
e x e n u t i o n	← substitute n by c
e x e c u t i o n	

Path from *intention* to *execution*.

$$D[i, j] = \min \begin{cases} D[i-1, j] + \text{del-cost}(\text{source}[i]) \\ D[i, j-1] + \text{ins-cost}(\text{target}[j]) \\ D[i-1, j-1] + \text{sub-cost}(\text{source}[i], \text{target}[j]) \end{cases}$$

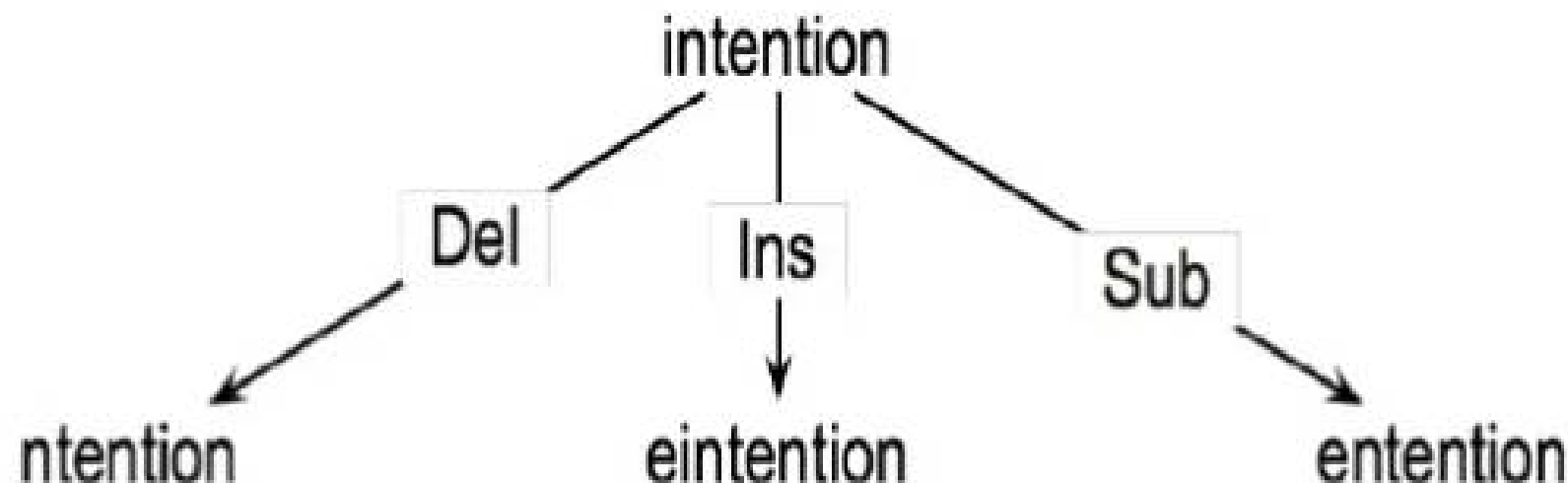
$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 \\ D[i, j-1] + 1 \\ D[i-1, j-1] + \begin{cases} 2; & \text{if } \text{source}[i] \neq \text{target}[j] \\ 0; & \text{if } \text{source}[i] = \text{target}[j] \end{cases} \end{cases}$$

Defining Min Edit Distance

- For two strings
 - X of length n
 - Y of length m
- We define $D(i,j)$
 - the edit distance between $X[1..i]$ and $Y[1..j]$
 - i.e., the first i characters of X and the first j characters of Y
 - The edit distance between X and Y is thus $D(n,m)$

How to find the Min Edit Distance?

- Searching for a path (sequence of edits) from the start string to the final string:
 - **Initial state:** the word we're transforming
 - **Operators:** insert, delete, substitute
 - **Goal state:** the word we're trying to get to
 - **Path cost:** what we want to minimize: the number of edits



Minimum Edit Distance

I N T E * N T I O N

| | | | | | | | |

* E X E C U T I O N

d s s i s

d-> delete

s-> substitution

i-> insert

- If each operation has cost of **1**, then Distance between these is **5**
- If substitution operation cost **2**, then Distance between them is **8**
 - The gap between **intention** and **execution**, for example, is **5** (delete an i, substitute e for n, substitute x for t, insert c, substitute u for n).
3 substitution + **1** insert + **1** delete = **5**

Minimum Edit Distance

- The minimum edit distance between two strings.
- It is the minimum number of editing operations.
 - Insertion
 - Deletion
 - Substitution
- Needed to transform one into the other.

How similar are two strings?

- Spell correction
 - The user typed “graffe”
 - Which is closest?
 - graf
 - graft
 - grail
 - giraffe

- Computational Biology
 - Align two sequences of nucleotides
AGGCTATCACCTGACCTCCAGGCCGATGCCC
TAGCTATCACGACCGCGGTCGATTGCCCCGAC
 - Resulting alignment:
–AGGCTATCACCTGACCTCCAGGCCGA--TGCCC---
TAG-CTATCAC--GACCGC--GGTCGATTGCCCCGAC

- Also for Machine Translation, Information Extraction, Speech Recognition

Basic Text Processing

Minimum Edit Distance

Summary

- Regular expressions play a surprisingly large role
 - Sophisticated sequences of regular expressions are often the first model for any text processing
- For many hard tasks, we use machine learning classifiers
 - But regular expressions are used as features in the classifiers
 - Can be very useful in capturing generalizations

Errors cont.

- In NLP we are always dealing with these kinds of errors.
- Reducing the error rate for an application often involves two antagonistic efforts:
 - Increasing accuracy or precision (minimizing false positives)
 - Increasing coverage or recall (minimizing false negatives).

- The process we just went through was based on fixing two kinds of errors
 - Matching strings that we should not have matched (there, then, other)
 - False positives (Type I)
 - Not matching things that we should have matched (The)
 - False negatives (Type II)

Example:

- Suppose we wanted to write a RE to find cases of the English article “the”. A simple (but incorrect) pattern might be:

/the/

- One problem is that this pattern will miss the word when it begins a sentence and hence is capitalized (i.e., The). This might lead us to the following pattern:

/[tT]he/

- But we will still incorrectly return texts with the embedded in other words (e.g., other or theology).
- So we need to specify that we want instances with a word boundary on both sides:

/\b[tT]he\b/

Regular Expressions: Boundary Anchors `\b` `\B`

- There are also two other anchors: `\b` matches a word boundary, and `\B` matches a non-boundary.
- For the purposes of a regular expression, a “word” is defined as any sequence of digits, underscores, or letters.
- Examples:
 - `/\bthe\b/` matches the word “the” but not the word “other”.
 - `/\b99\b/` will match the string 99 in “There are 99 bottles of juice on the wall” (because 99 follows a space and precedes a space) but not 99 in “There are 299 bottles of juice on the wall” (since 99 follows a number). But it will match 99 in “\$99” (since 99 follows a dollar sign (\$), which is not a digit, underscore, or letter).
- What will be the results of using the other anchor: `\B` in the previous examples knowing that it matches a non-word boundary?

Regular Expressions: Anchors [^] ^{\$}

- **Anchors** are special characters that anchor regular expressions to particular places in a string.
- The caret ([^]) matches the **start of a line**.
 - The pattern [^]The/ matches the word “The” only at the start of a line.
- The dollar sign ^{\$} matches the **end of a line**.
 - [^]The dog\sup>\$/ matches a line that contains only the phrase “The dog”.

Pattern	Matches
[^] [A-Z]	<u>P</u> alo Alto
[^] [[^] A-Za-z]	<u>1</u> <u>"</u> Hello”
\. ^{\$}	The end <u>.</u>
. ^{\$}	The end <u>?</u> The end <u>!</u>

Regular Expressions: ? * + .

- The question mark (?) Symbol means **zero** or **one** instance of the preceding character.
- The **Kleene** asterisk (*) symbol means **zero** or **more** occurrences of the preceding character.
- The **Kleene** (+) symbol means **one** or **more** occurrences of the preceding character.
- The period (.) symbol is a **wildcard** expression that matches **any single** character it represents within the pattern (except a carriage return).

Pattern	Matches	
colou?r	Optional previous char	Color Colour
oo*h!	0 or more of previous char	oh! ooh! ooooh! oooooh!
o+h!	1 or more of previous char	oh! ooh! ooooh! oooooh!
baa+	1 or more of previous char	baa baaa baaaa baaaaa
beg.n	Only 1 character	begin begun begun beg3n

Regular Expressions: More Disjunction

- Woodchucks is another name for groundhog!
- The **pipe** `|` symbol for **disjunction**

Pattern	Matches
<code>groundhog woodchuck</code>	groundhog woodchuck
<code>yours mine</code>	yours mine
<code>a b c</code>	= <code>[abc]</code>
<code>[gG]roundhog [Ww]oodchuck</code>	groundhog woodchuck Groundhog Woodchuck

Regular Expressions: Negation in Disjunction

- **Negations** can be applied using the **caret** `^` symbol
 - **Caret** means negation only when **first** in []

Pattern	Matches	Example Patterns Matched
<code>[^A-Z]</code>	Not an upper case letter	O <u>y</u> fn pripetchik
<code>[^Ss]</code>	Neither 'S' nor 's'	<u>I</u> have no exquisite reason"
<code>[^e^]</code>	Neither e nor ^	Look here <u>e</u>
<code>a^b</code>	The pattern <i>a</i> caret <i>b</i>	Look up <u>a^b</u> now

Regular Expressions: Disjunctions

- Use **dash** - inside brackets to specify any **one** character in a **range**.

Pattern	Matches	Example Patterns Matched
[A-Z]	An upper case letter	<u>D</u> renched Blossoms
[a-z]	A lower case letter	<u>m</u> y beans were impatient
[0-9]	A single digit	Chapter <u>1</u> : Down the Rabbit Hole

Regular Expressions

- Regular expressions are **case sensitive**. This means that the pattern `/woodchucks/` will not match the string “Woodchucks”.
 - We can solve this by using **square braces** `[]`
 - The string of characters inside the braces `[]` specifies a **disjunction** of characters to match.

RE	Match	Example Patterns
<code>/[wW]oodchuck/</code>	Woodchuck or woodchuck	“ <u>W</u> oodchuck”
<code>/[abc]/</code>	‘a’, ‘b’, <i>or</i> ‘c’	“In uomini, in soldati”
<code>/[1234567890]/</code>	any digit	“plenty of <u>7</u> to 5”

The use of the brackets `[]` to specify a disjunction of characters.

Regular Expressions

- A formal language for specifying text strings.
- Formally, a regular expression is an algebraic notation for characterizing a set of strings.
- A regular expression search function will search through a **corpus**, returning all texts that match a **pattern**.
 - The simplest kind of regular expression is a sequence of simple characters.
 - For example:

RE	Example Patterns Matched
/woodchucks/	“interesting links to <u>woodchucks</u> and lemurs”
/a/	“ <u>M</u> ary Ann stopped by Mona’s”
/!/	“You’ve left the burglar behind again!” said Nori